Contents

Guide 1 — Web Scraping (how it works, the tools, file by file)

This is the first of several guides. It explains, in plain language, the **web scraping** part of Mobile Match Algeria: what it is, the tools we chose and why, how the whole thing works step by step, and what each file does. Read it once and you’ll be able to explain and defend this part of the project confidently.

---

1. What is web scraping (in one minute)

The three operators (Djezzy, Ooredoo, Mobilis) publish their mobile plans on their websites, but **none of them offers an API** (a clean data feed a program could ask). So to keep our database up to date automatically, our program acts like a visitor: it **opens the operator pages, reads the HTML, and pulls out the numbers** (price, data, minutes, SMS, validity). That is web scraping — automated reading of public web pages.

We only read **public marketing pages** (the same pages anyone sees in a browser), we respect delays, and we collect **no personal data**. This is the legal, ethical way to do it.

---

2. The tools we use, and why

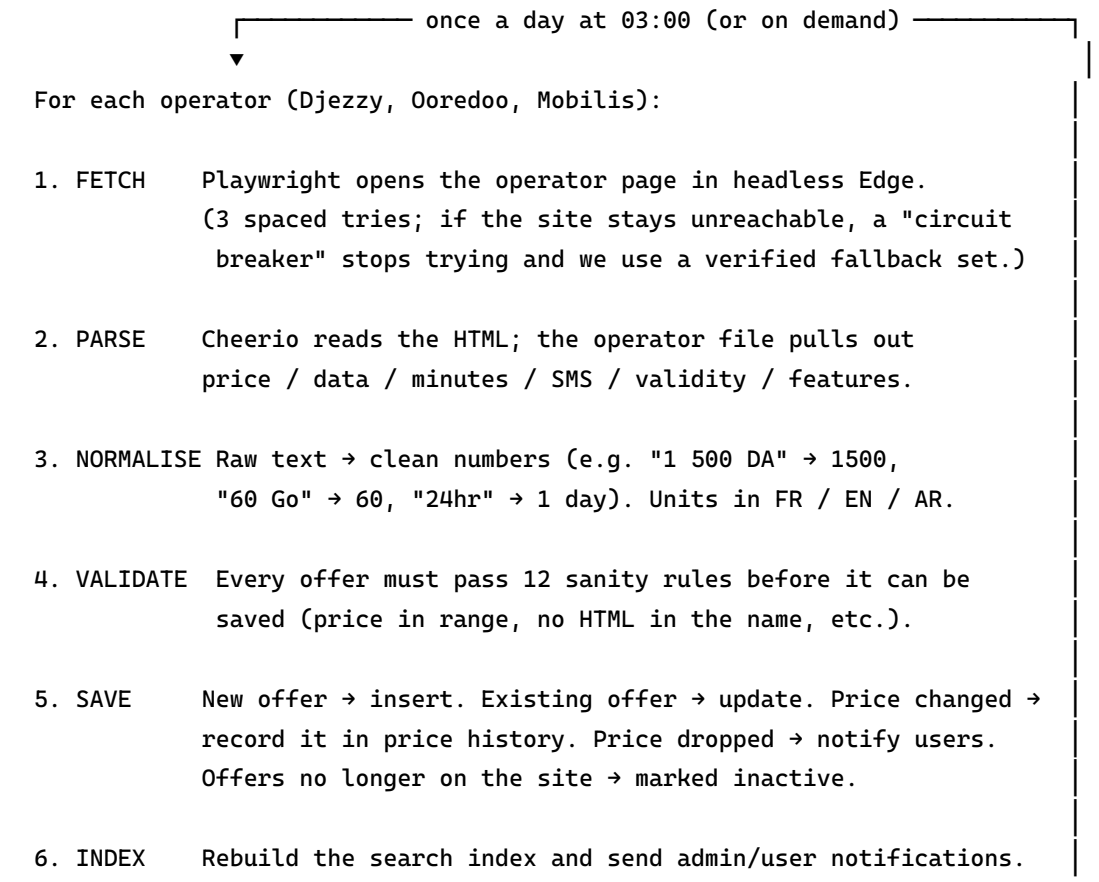
Tool	Role in our project	Why this one
Playwright	Drives a real web browser from code (opens pages, waits, reads the HTML).	The operator sites have anti-bot protection that **blocks plain HTTP requests**. A real browser passes; a simple `fetch()` gets rejected. Playwright is the modern, reliable choice for this.

Tool	Role in our project	Why this one
Microsoft Edge (headless)	The actual browser Playwright drives, with no window shown.	Using a *real* installed browser is the **least detectable** option. If Edge isn't installed, the code automatically falls back to Playwright's bundled Chromium.
Cheerio	Reads the downloaded HTML and lets us pick out fields with CSS selectors (like jQuery, but on the server).	Once the browser hands us the HTML, we no longer need the heavy browser to *parse* it. Cheerio is tiny and fast for that job.
node-cron	Runs the scrape automatically once a day.	Simple, runs inside our own Node server — no extra service to host.

One sentence to remember: *Playwright + real headless Edge fetch the page, Cheerio parses it, and node-cron runs the whole thing daily.*

---

3. How it works, step by step



A healthy run for all three operators finishes in a few seconds and ends with ~98 active offers in the database.

---

4. The files, one by one

All scraping code lives in src/lib/scraper/. Here is each file and what it does.

fetch.ts — the browser fetcher (the "hands")

This is the only file that touches the browser. - Launches **headless Microsoft Edge** through Playwright (falls back to bundled Chromium if Edge is missing). - Sets a realistic browser identity (French locale, Algiers timezone, normal user-agent) and hides the obvious "I am a robot" signal. - `fetchPage(url)` tries each page up to **3 times** with spaced waits (0s, 7s, 18s) — operator servers (Djezzy especially) randomly drop connections, and spreading the tries catches a good moment. - A **circuit breaker**: after repeated failures it stops hammering a dead site and lets the operator fall back to verified data quickly instead of hanging.

validate.ts — the shared parsers + the safety gate

The single source of truth for turning messy text into clean data. - **Parsers**: `parseGB`, `parsePriceDA`, `parseValidityDays` — each understands French (Go/Mo, jours/mois), English (GB/MB), and Arabic (جيغا/ميغا، يوم/شهر). They are *unit-anchored*: `parseGB("60")` returns 0; only `"60 Go"` returns 60. This prevents misreading a price as a data amount. - `cleanText` / `cleanFeatureText` — strip HTML and junk, keep real feature lines. - `validateOffer` — checks **12 rules** (price between 10 and 50 000 DA, data not absurd, no `<script>` in the name, validity 1–365 days, a valid type, and cross-checks that catch parsing bugs). Nothing reaches the database without passing this gate.

djezzy.ts, ooredoo.ts, mobilis.ts — one per operator

Each file knows the HTML shape of *its* operator's website. - It asks `fetch.ts` for each page, then uses **Cheerio** selectors (e.g. `article.price-card`, `li.feature-item`) to read out the fields, calling the shared parsers from `validate.ts`. - Sentinels: a value of `-1` **means "unlimited"** and `0` **means "none"** for minutes and SMS (so "Unlimited Djezzy SMS" is stored as `-1`, and the human label is rebuilt later for display). - **Fallback dataset**: if the live site is unreachable or changed its HTML, the file returns a **hand-verified set of offers** so the app never goes blank. (For Mobilis Revolution and Djezzy iZZY the prices are inside *images*, so those few are always hand-maintained — see the "stale data" reminder below.)

index.ts — the orchestrator (the "brain")

Runs the whole job and is the only file that writes to the database. - Runs the three operator scrapers in turn. - Re-validates every offer (defense in depth), then **de-duplicates** by slug (carousels can output the same plan twice). - **Saves**: inserts new offers, updates existing ones; when a price changes it writes a row to **price history**; when a price *drops* it notifies users. - **Deactivates stale offers**: any active offer that's no longer on the site gets marked inactive — with a safety guard that *skips* this if the run returned suspiciously few offers (so a broken run can't wipe the catalogue). - Rebuilds the **search index**, then sends notifications: a summary to admins ("Scrape complete — N new"), failure alerts to admins, and personalised "match found" alerts to users. - **Stale-data reminder**: the image-only offers can't be auto-checked, so after 45 days the orchestrator reminds admins (at most once a week) to re-verify them.

../instrumentation.ts — the automatic starter

Next.js runs this once when the server boots. It: - registers the **daily cron at 03:00 Africa/Algiers**, - rebuilds the search index on boot, - runs a scrape **on startup if the DB is empty or the last scrape was >23h ago**,

- schedules a **weekly cleanup** of old scrape logs. This is why scraping "just works" with `npm run dev` — no separate process needed.

../cron.ts — optional standalone runner

The same daily schedule, but as a separate process you can run by hand. Only needed if you don't want it inside the web server.

../../app/api/cron/route.ts and ../../app/api/admin/scrape/route.ts — manual triggers

- `POST /api/cron?secret=…` lets an external scheduler trigger a scrape (protected by a secret).
- The admin route lets a logged-in admin run a scrape **on demand** from the Admin panel button.

tests/ — proof it works

`parsers.test.ts` and `validate.test.ts` test the parsing and validation against **saved real HTML samples** (fixtures), so we can prove the scraper behaves correctly without hitting the live sites.

---

5. The smart parts worth mentioning at the defense

- **Why a real browser?** Plain HTTP is blocked by anti-bot; a genuine headless Edge passes. (This is the static-vs-dynamic choice from Chapter 1.)
- **Retries + circuit breaker** → resilient to flaky operator servers without hanging.
- **Verified fallback dataset** → the site is never empty even when an operator is down or redesigns its page.
- **Unit-anchored parsers + a 12-rule validation gate** → no garbage ever reaches the database (this directly fixed an early "price read as data" bug).
- **Per-operator deactivation with a safety guard** → the catalogue stays in sync with reality, but one broken run can't erase it.
- **Trilingual parsing (FR / EN / AR)** → the operators mix languages, and we read all three.

---

6. Likely jury questions (and short answers)

- **"Is scraping legal?"** — We read only public pages, obey `robots.txt` and rate limits, identify ourselves, and store no personal data. Same data any visitor sees.
- **"What if a site changes?"** — The operator file fails gracefully and the verified fallback dataset takes over; tests on saved HTML catch breakage early.
- **"Why not just use an API?"** — No Algerian operator publishes one; scraping is the standard method when data is only in HTML.
- **"How often does it run?"** — Automatically every day at 03:00 (Algiers), plus on startup if data is missing/old, plus on demand from the Admin panel.

Next guide: the recommendation engine (the formula, the tiers, the files). Tell me when you want it and I'll write `02-recommendation-engine.md` *the same way.*